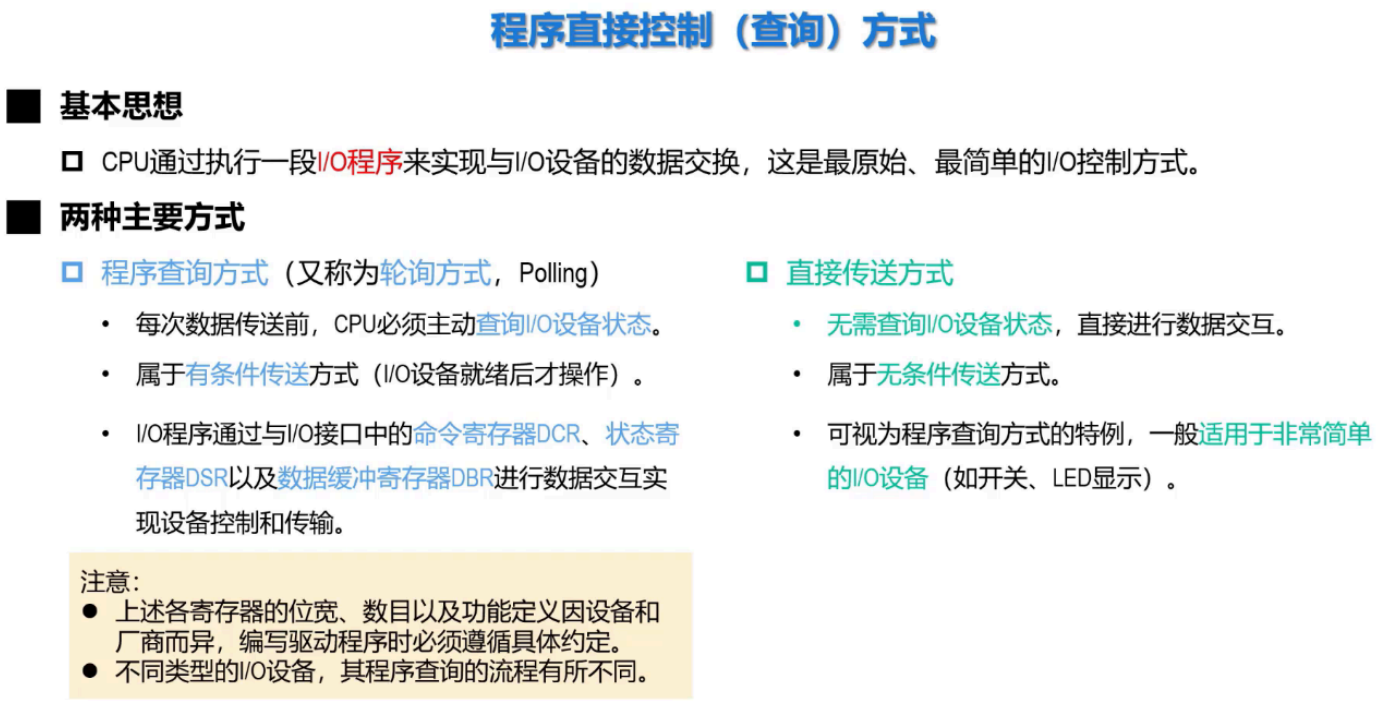
8.5 程序直接控制方式

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.5）｜重要度 ★★★★｜常见考法：选择题（查询流程、DSR/DBR、忙等待 vs 定时轮询、MIPS/x86 指令），偶作流程填空

一句话核心：CPU 主动轮询 DSR，设备就绪后再经 DBR 传送数据；最简单，但 CPU 空转严重

一、基本思想与两种方式



基本思想：CPU 通过执行一段 I/O 程序实现与 I/O 设备的数据交换，是最原始、最简单的 I/O 控制方式

方式	是否查询	传送条件	适用
程序查询方式（轮询，Polling）	每次数据传送前，CPU 必须主动查询设备状态	有条件传送：设备就绪后才操作	大多数慢速/中速设备
直接传送方式	无需查询设备状态，直接进行数据交互	无条件传送	非常简单的 I/O 设备，如开关、LED 显示

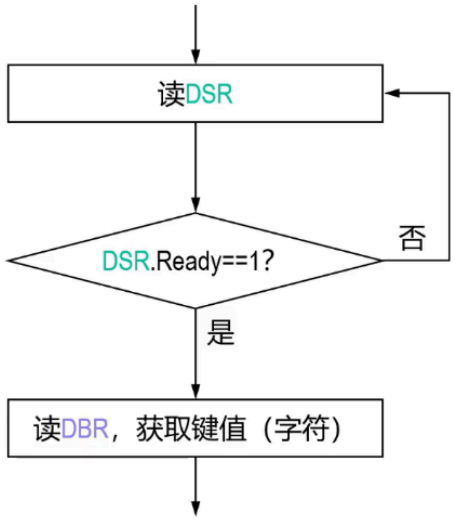
- 程序查询方式通过 I/O 接口中的命令寄存器 DCR、状态寄存器 DSR、数据缓冲寄存器 DBR 进行数据交互
- 注意：寄存器的位置、数目、功能定义因设备和厂商而异，编写驱动程序时必须遵循具体约定
- 不同设备的查询流程不同，不能照搬同一个流程

二、简单输入设备：键盘

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘

- 没有命令寄存器DCR
- 当按下按键时，键盘控制器执行以下操作：
  - 将被按下的按键的键值（ASCII）存放在键盘的数据缓冲寄存器DBR中。
  - 同时将键盘的状态寄存器DSR中的“Ready”标志位置为1，表示当前有按键数据。
- 当CPU需要从键盘获取按键数据时，则通过总线读取DSR寄存器，以便查询DSR中的Ready标志位：
  - 若为0，则继续查询直至该位为1；
  - 若为1，则通过总线读取DBR中的键值，完成操作后DSR中的Ready标志位自动清零。



如果需要连续获取多个按键，需要给上述流程加上循环控制。

键盘没有命令寄存器 DCR，流程比复杂设备简单：

1. 按下按键时，键盘控制器把键值（ASCII）存入数据缓冲寄存器 DBR
2. 同时把状态寄存器 DSR 中的 Ready 标志位置 1，表示当前有按键数据
3. CPU 通过总线读 DSR，查询 Ready 标志
4. Ready = 0 → 继续查询，直到为 1
5. Ready = 1 → 读 DBR，获取键值；完成操作后 DSR 的 Ready 标志自动清零
6. 如果要连续获取多个按键，需要给上述流程加上循环

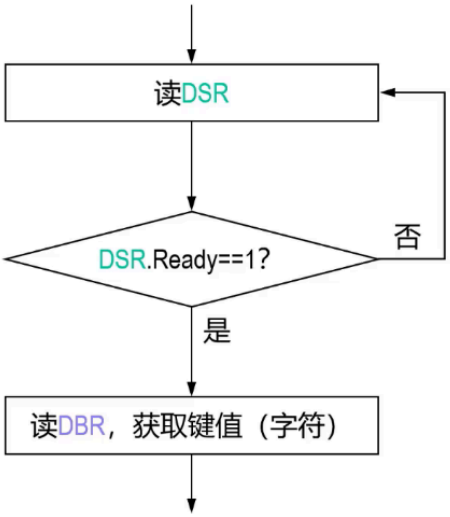
C 风格伪代码：

```
// 假设 DSR.Ready 为状态寄存器最低位，DBR 为数据缓冲寄存器
while (DSR.Ready == 0) { /* 忙等待，反复查询 */ }
key = DBR;   // 读取键值；读完后 Ready 自动清零
```

三、MIPS 统一编址实现

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘



MIPS架构中I/O接口的编址采用统一编址（内存映射编址）方式，使用访存指令访问，假设：

- 键盘设备中的8位设备状态寄存器DSR的内存映射地址为0xffff0000，最低位为Ready标志位。
- 键盘设备中的8位数据缓冲寄存器DBR的内存映射地址为0xffff0004。

```
        lui  $t1, 0xffff0000    # 载入键盘状态寄存器DSR的内存映射地址到$t1
keyPoll: lbu  $t4, ($t1)        # 载入DSR的值到$t4
        andi $t4, $t4, 1        # 获取DSR的Ready标志位（最低位）
        beq  $t4, $0, keyPoll   # 若Ready==0，则跳转到keyPoll处继续查询
        lui  $t1, 0xffff0004    # 载入键盘数据缓冲寄存器DBR的内存映射地址到$t1
        lbu  $s2, ($t1)         # 读取DBR中的字符数据（键值）到$s2
```

MIPS 采用统一编址（内存映射编址），使用访存指令访问端口。假设键盘状态寄存器 DSR 的内存映射地址为 `0xFFFF0000`，最低位为 Ready 标志；数据缓冲寄存器 DBR 的内存映射地址为 `0xFFFF0004`

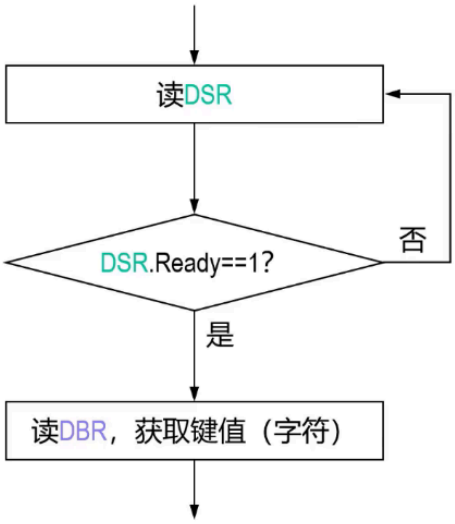
```
        lui   $t1, 0xFFFF0000      # 装入键盘状态寄存器 DSR 的内存映射地址到 $t1
keyPoll:
        lw    $t0, 0($t1)          # 读 DSR 的值到 $t0
        andi  $t0, $t0, 1          # 取 DSR 的 Ready 标志位（最低位）
        beq   $t0, $zero, keyPoll  # 若 Ready=0，则跳回 keyPoll 继续查询
        lui   $t1, 0xFFFF0004      # 装入键盘数据缓冲寄存器 DBR 的内存映射地址到 $t1
        lw    $t2, 0($t1)          # 读取 DBR 中的字符数据（键值）到 $t2
```

- `lui` 装入地址高 16 位，`lw` 从内存映射端口取数，`andi` 屏蔽出最低位，`beq` 构成查询循环
- 统一编址下，CPU 不需要专门的 I/O 指令，用普通访存指令即可

四、x86 独立编址实现

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘



x86架构中I/O接口的编址采用独立编址（端口映射编址）方式，使用专用的I/O指令访问，假设：

- 键盘设备中的8位设备状态寄存器DSR的端口地址为0x0000，最低位为Ready标志位。
- 键盘设备中的8位数据缓冲寄存器DBR的端口地址为0x0004。

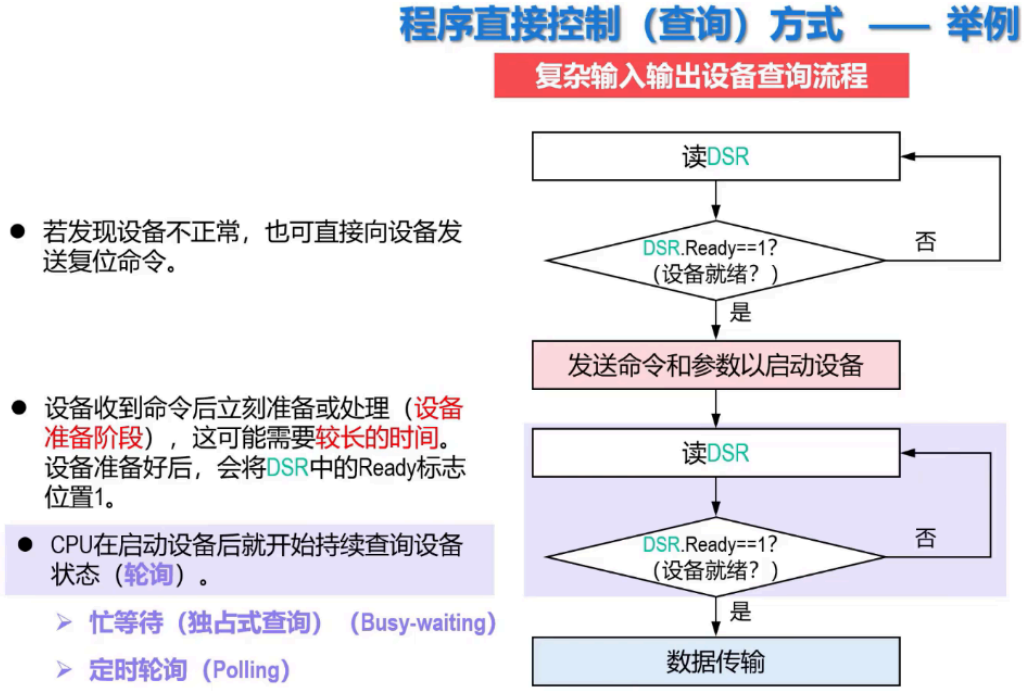
```
        MOV   DX, 0x0000     # 载入键盘状态寄存器DSR的端口地址到DX
keyPoll: IN   AL, DX         # 载入DSR的值到AL
        TEST  AL, 1          # 测试DSR的Ready标志位（最低位）
        JZ    keyPoll        # 若Ready==0，则跳转到keyPoll处继续查询
        MOV   DX, 0x0004     # 载入键盘数据缓冲寄存器DBR的端口地址到DX
        IN    AL, DX         # 读取DBR中的字符数据（键值）到AL
```

x86 采用独立编址（端口映射编址），使用专用 I/O 指令。假设键盘状态寄存器 DSR 的端口地址为 `0x0060`，最低位为 Ready 标志；数据缓冲寄存器 DBR 的端口地址为 `0x0064`

```
        MOV  DX, 0x0060      ; 装入键盘状态寄存器 DSR 的端口地址到 DX
keyPoll:
        IN   AL, DX          ; 读 DSR 的值到 AL
        TEST AL, 1           ; 测试 DSR 的 Ready 标志位（最低位）
        JZ   keyPoll         ; 若 Ready=0，则跳回 keyPoll 继续查询
        MOV  DX, 0x0064      ; 装入键盘数据缓冲寄存器 DBR 的端口地址到 DX
        IN   AL, DX          ; 读取 DBR 中的字符数据（键值）到 AL
```

- `IN` 是专用 I/O 指令，端口地址与主存地址分属两个空间
- `TEST` 只影响标志位，`JZ` 根据零标志跳转，构成查询循环
- 与 MIPS 版本对照：同样的查询逻辑，统一编址用 `lw`，独立编址用 `IN`

五、复杂 I/O 设备的查询流程

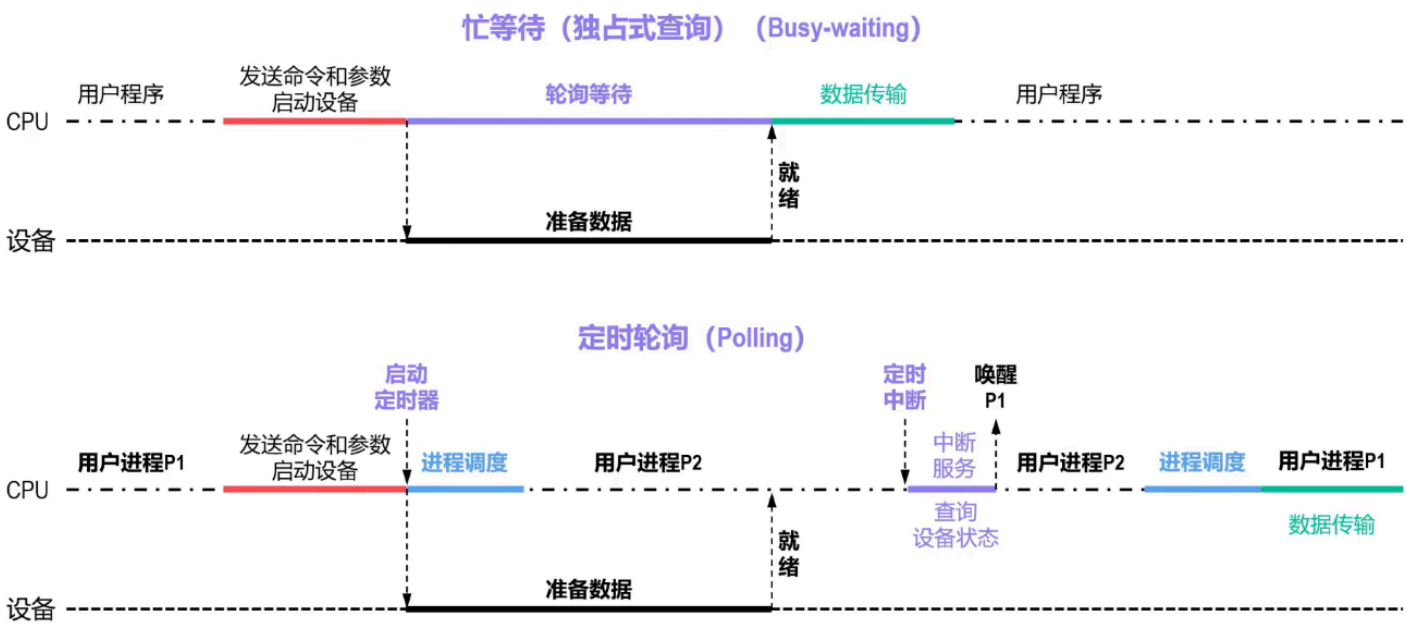


复杂设备通常有 DCR，流程多出“下发命令 + 设备准备”两步：

1. 读 DSR，判断设备是否正常；若异常，可向设备发送复位命令
2. 发送命令和参数，启动设备
3. 设备收到命令后立即准备或处理（设备准备阶段），这可能要较长时间
4. 设备准备好后，把 DSR 中的 Ready 标志位置 1
5. CPU 从启动设备后就开始持续查询设备状态
6. Ready = 1 后，进行数据传输

查询等待有两种策略：忙等待（独占式查询，Busy-waiting）与定时轮询（Polling）

六、忙等待 vs 定时轮询



维度	忙等待（独占式查询）	定时轮询
CPU 行为	连续读 DSR，原地循环	启动定时器，转去执行其他进程；定时中断到来再查询
CPU 利用率	低，轮询等待浪费大量 CPU 时间	较高，设备准备数据期间 CPU 可执行其他任务
响应及时性	设备一就绪立刻发现	取决于定时时间间隔
额外开销	纯轮询开销	定时中断、进程调度、中断服务开销
适用	单任务操作系统；超高速设备（等待时间短）	多任务操作系统；设备无中断请求功能；慢速设备

- **轮询等待**浪费了大量CPU时间。
- 一般在**单任务操作系统**中可采用独占式查询，因为CPU在轮询等待过程中不需要执行其他任务。
- 对于**超高速设备**，由于设备响应快，相应的轮询等待时间短，独占式查询也可以接受。

- 轮询等待浪费大量 CPU 时间
- 单任务操作系统中可采用独占式查询，因为 CPU 在轮询等待过程中不需要执行其他任务
- 对于超高速设备，设备响应快，轮询等待时间短，独占式查询也可以接受

定时轮询的优点

- 设备准备数据的过程中，CPU可执行其他任务。
- 有效避免了轮询等待对CPU时间的大量浪费，但中断服务还需要占用一定的CPU时间。
- 适用于外部设备自身没有中断请求功能的多任务操作系统。

- 设备准备数据的过程中，CPU 可执行其他任务
- 有效避免了轮询等待对 CPU 时间的大量浪费，但中断服务还需要占用一定的 CPU 时间
- 适用于外部设备自身没有中断请求功能的多任务操作系统

定时时间间隔的取舍

- 定时时间间隔的选择是关键问题：
  - 太短：中断频繁，进而频繁进行**中断服务**，**开销浪费大。**
  - 太长：I/O设备的**数据可能得不到及时处理。**
- 为了提高CPU利用率，并且确保I/O设备的数据得到及时处理而不丢失，需要合理设置定时时间间隔。
  - 例如：键盘设备和鼠标设备的输入速度比CPU慢得多，因此，只要以比人的反应速度快的频率进行定时轮询，就可以确保设备数据不丢失。

- 太短：中断频繁，频繁进行中断服务，开销浪费大
- 太长：I/O 设备的数据可能得不到及时处理
- 需要合理设置定时时间间隔，既提高 CPU 利用率，又确保设备数据及时处理而不丢失
- 例如键盘和鼠标的输入速度比 CPU 慢得多，只要以比人的反应速度快的频率进行定时轮询，就能确保数据不丢失

七、适用场景

- 慢速设备比较适合定时轮询，而快速设备的定时轮询开销大。
- 程序直接控制（查询）方式的I/O接口设计简单，容易实现，常见于早期单任务操作系统中。
- 现代多任务操作系统在多任务引导成功之前，必须使用程序直接控制（查询）方式与I/O设备进行数据交互（如键盘、磁盘、显示器等都必须支持程序直接控制方式）。

- 慢速设备比较适合定时轮询，快速设备的定时轮询开销大
- 程序直接控制（查询）方式的 I/O 接口设计简单，容易实现，常见于早期单任务操作系统
- 现代多任务操作系统在多任务引导成功之前，必须使用程序直接控制（查询）方式与 I/O 设备进行数据交互（如键盘、磁盘、显示器等都必须支持程序直接控制方式）

八、易错点

- 程序查询不一定只适合慢速设备：超高速设备等待时间短，独占式查询也可接受
- 定时轮询不是“设备主动请求”：设备仍然被动，只是 CPU 用定时器周期性来查

- 忙等待与定时轮询的区别：前者原地连续查，后者用定时中断插入查询
- 直接传送方式是无条件传送，不需要查询；不要把“直接传送”与“DMA 直接存储器存取”混为一谈
- 键盘例子中 DSR 的 Ready 位在读走 DBR 后自动清零；不同设备的清零方式可能不同
- 统一编址用访存指令，独立编址用 IN/OUT 指令；不要记反

九、考法透视

选择题常见设错点

1. 把程序查询方式说成“设备主动通知 CPU” → 这是中断驱动方式
2. 认为程序查询只适合慢速设备 → 超高速设备也可用独占式查询
3. 认为定时轮询没有中断开销 → 定时器中断、调度、ISR 都有开销
4. 把“直接传送方式”与 DMA 混淆 → 前者是无条件程序传送，后者是硬件成块搬运
5. 记反 MIPS/x86 的编址与指令对应关系

流程题

- 给定查询流程图，补全“读 DSR → 判断 Ready → 读 DBR”的顺序
- 给定 MIPS/x86 代码，说明每步作用或填空
- 比较忙等待与定时轮询的 CPU 利用率和响应及时性

十、知识关联

- 前置：8.3 IO接口（DSR、DBR、DCR 与编址方式）
- 后续：8.6 中断驱动方式概述（用中断消除轮询）、8.12 DMA控制方式（用硬件消除逐字传送）
- OS 视角：进程阻塞/唤醒、定时器中断、调度开销
- 全章导航：8.0 导航与术语

十一、费曼自测

1. 程序查询方式的基本步骤是什么 → CPU 读 DSR 查 Ready；未就绪继续查；就绪后读/写 DBR 完成一次传送；连续传送要循环
2. 忙等待和定时轮询的本质区别是什么 → 忙等待原地连续查，CPU 空转；定时轮询用定时器中断周期性插入查询，等待期间 CPU 可执行其他任务
3. 为什么程序查询不一定只适合慢速设备 → 超高速设备准备数据极快，轮询等待很短，独占式查询的开销也可以接受
4. MIPS 与 x86 访问端口分别用什么指令 → MIPS 统一编址，用 lw/sw 等访存指令；x86 独立编址，用 IN/OUT 专用指令
5. 定时轮询的时间间隔太长或太短分别有什么问题 → 太短：中断频繁、开销大；太长：设备数据可能丢失、响应不及时